Projet de Programmation
Master Informatique
2025–2026

université de BORDEAUX

Quoridor – Java

Rapport final

Abdeldjabar Ismail Abdraman, Cengiz Vasseur, Joris Douillet, Khadidja Ahmat Hassan, Matthieu Pourageaud

8 avril 2026

1 Les Fonctionnalités étendues

1.1 Fonctionnalité 1 : Réseau client/serveur et gestion multi-clients

1.1.1 Besoin initial

L'un des objectifs du projet était d'ajouter une dimension réseau au jeu Quoridor afin de permettre à plusieurs joueurs d'interagir à distance. Dans une première étape, il s'agissait de mettre en place un serveur simple capable d'accepter une connexion cliente, de répondre à des commandes élémentaires telles que `PING` et `QUIT`, et de permettre au client de tester la connexion.

Dans une seconde étape, l'objectif a été étendu à la gestion simultanée de plusieurs clients et de plusieurs parties en parallèle. Le serveur devait alors être capable de conserver l'état de plusieurs sessions de jeu, de créer des parties entre joueurs disponibles, de valider les coups reçus à distance et de transmettre les actions aux autres joueurs concernés.

L'intérêt de cette fonctionnalité est double. D'une part, elle transforme le jeu local en application distribuée selon une architecture client/serveur. D'autre part, elle centralise la validation des coups côté serveur, ce qui évite qu'un client applique localement des actions non vérifiées.

1.1.2 Choix de conception

Le choix retenu dans le projet est une architecture client/serveur. Le serveur joue le rôle d'autorité centrale : il accepte les connexions, reçoit les commandes, valide les actions envoyées par les clients et conserve l'état officiel des parties.

L'implémentation repose sur deux niveaux principaux. La classe `GameServer` correspond à un serveur simple, conçu pour la première étape du réseau. Elle gère une seule connexion cliente, répond à des commandes de base comme `PING` et `QUIT`, lance un diffuseur UDP et met en place un timeout d'une minute. La classe `MultiGameServer` constitue l'extension multi-clients.

Le protocole choisi est volontairement simple : les messages échangés sont des chaînes ASCII terminées par un retour à la ligne. Les commandes principales sont `HELLO`, `PING`, `QUIT` et `MOVE`, avec des réponses comme `WELCOME`, `PONG`, `BYE`, `OK`, `ERROR`, `GAME_START`, `OPPONENT_MOVE` et `GAME_OVER`.

Pour l'annonce de présence du serveur, le projet utilise une classe `ServerBroadcaster`. Côté client, la méthode `serverList()` de `GameClient` écoute les messages UDP afin de détecter les serveurs disponibles.

Enfin, la validation des coups ne repose pas sur le client. Lorsqu'un joueur envoie un coup sous forme textuelle, celui-ci est converti en objet métier `Move` par `NetworkMoveParser`, puis transmis au moteur de jeu côté serveur.

1.1.3 Structures de données utilisées

La partie réseau s'appuie sur plusieurs structures de données importantes.

Dans `MultiGameServer`, les clients sont stockés dans une structure `Map<String, ClientSession>`. Chaque client possède un identifiant unique (C1, C2, etc.).

Les parties actives sont stockées dans `Map<String, ServerGameSession>`.

Le tableau des scores repose sur `Map<String, ScoreEntry>`.

La synchronisation repose sur un verrou utilisé avec `synchronized(lock)`.

Côté client, `GameClient` utilise une `SynchronousQueue<String>` pour gérer les réponses synchrones.

1.1.4 Algorithme mis en place

Le serveur ouvre une socket TCP, lance un thread UDP pour la diffusion et un thread de surveillance pour les timeouts.

Chaque connexion crée une `ClientSession`. Lorsqu'un client envoie une commande :

— `HELLO` : enregistre le joueur
— `PING` : réponse `PONG`
— `QUIT` : réponse `BYE`
— `MOVE` : traitement du coup

Pour un coup :

— Vérification du joueur et du tour
— Conversion en `Move`
— Validation par le moteur de jeu

Si le coup est valide :

— Envoi de `OK`
— Diffusion de `OPPONENT_MOVE`

Fin de partie :

— Mise à jour du score
— Envoi de `GAME_OVER WIN` ou `GAME_OVER LOSE`

1.1.5 Difficultés rencontrées

La principale difficulté de cette fonctionnalité a été la gestion de la concurrence. En effet, dans la version multi-clients, le serveur fonctionne avec plusieurs threads : un thread principal d'acceptation des connexions, un thread par client, un thread pour le diffuseur UDP et un thread pour la surveillance des timeouts. Sans mécanisme de protection, plusieurs accès simultanés aux listes de clients, de parties ou de scores auraient pu provoquer des incohérences. L'utilisation d'un verrou commun avec synchronized (lock) a donc été nécessaire.

Une deuxième difficulté a concerné la gestion des réponses synchrones et des messages asynchrones côté client. Certaines commandes, comme PING ou QUIT, attendent une réponse précise et immédiate. En parallèle, d'autres messages peuvent arriver à tout moment depuis le serveur, par exemple WELCOME, OPPONENT_MOVE ou GAME_OVER. Il a donc fallu concevoir un client capable de lire les messages en continu tout en permettant à certaines méthodes d'attendre une réponse particulière. Cette difficulté a été traitée dans GameClient avec un thread d'écoute et une file de synchronisation.

Une troisième difficulté a été l'intégration du protocole réseau avec la logique métier du jeu. Le serveur ne se contente pas de relayer les messages ; il doit interpréter un coup reçu sous forme textuelle, le convertir correctement en objet métier et vérifier sa légalité avec le moteur du jeu. Cela impose une bonne articulation entre les classes réseau et les classes du noyau du projet.

Enfin, la découverte de serveurs a demandé une conception séparant deux types d'échanges : le TCP, utilisé pour la communication fiable entre client et serveur pendant la partie, et l'UDP, utilisé pour l'annonce de présence du serveur. Dans l'état actuel du projet, cette partie existe bien mais reste encore partiellement finalisée.

1.1.6 Tests réalisés

La fonctionnalité réseau a été accompagnée de plusieurs tests unitaires.

La classe GameServerTest vérifie le fonctionnement du serveur simple. Les tests confirment qu'un client peut se connecter, qu'un message PING reçoit bien PONG, qu'une commande QUIT reçoit BYE, et qu'une commande inconnue reçoit une réponse générique.

La classe GameClientTest vérifie les comportements essentiels du client. Les tests contrôlent la réussite ou l'échec d'une connexion, l'affichage d'un message d'erreur lorsqu'aucune connexion n'est active, l'envoi correct des commandes HELLO, MOVE et QUIT, ainsi que la commande serverList() à l'aide d'un faux serveur UDP.

La classe MultiGameServerTest couvre une partie importante de la logique multi-clients. On y retrouve notamment des tests sur l'état du serveur, la liste des joueurs, le tableau des scores, la création de parties à deux ou quatre joueurs, le changement de statut des joueurs, la gestion des tours, les erreurs liées à un joueur absent ou indisponible, ainsi que certains comportements liés à la validation des coups, à leur diffusion et à la fin d'une partie.

D'autres tests existent également autour de classes connexes comme NetworkMoveParserTest, ServerBroadcasterTest, ClientSessionTest, ServerGameSessionTest et NetworkLauncherTest. Dans l'ensemble, ces tests montrent que la partie réseau n'a pas seulement été commencée, mais qu'elle a réellement été structurée et vérifiée.

1.1.7 Résultat final

Au stade actuel du projet, la fonctionnalité réseau est bien avancée et constitue une partie importante du système. Le projet contient un serveur simple, ainsi qu'un serveur multi-clients permettant de gérer plusieurs joueurs simul-

tanément, de créer des parties à deux ou quatre joueurs, de maintenir un tableau des scores et de valider les coups à distance avant leur diffusion.

Le résultat obtenu montre que l'architecture du projet est suffisamment modulaire pour réutiliser le moteur de jeu existant dans un contexte réseau. Cette réutilisation est un point fort, car elle évite de dupliquer les règles du jeu entre le mode local et le mode distant.

Cependant, l'implémentation actuelle ne couvre pas encore parfaitement l'ensemble du cahier des charges. La gestion multi-clients et multi-parties est globalement réalisée et fonctionnelle. En revanche, la découverte complète des serveurs sur le réseau local reste partielle. Par ailleurs, même si des commandes réseau existent en mode ligne de commande et dans l'interface graphique, leur intégration n'est pas encore totalement finalisée dans toutes les interfaces. Il est donc plus juste de dire que la partie la plus avancée du réseau est satisfaisante dans l'ensemble, mais que certains points restent à améliorer.

1.2 Fonctionnalité 2 AI :

1.2.1 Introduction et organisation du module IA

L'intelligence artificielle est la composante du projet qui cherche le meilleur coup à jouer selon une situation du jeu. Quoridor est un jeu où chaque décision peut avoir des conséquences bien plus loin dans la partie, comme par exemple la pose d'un mur censé bloquer l'adversaire et qui finalement se retrouve à nous bloquer nous. L'évaluation d'une position sera complexe car il faut donc prévoir à l'avance si le coup que nous jouons peut être désavantageux pour nous des dizaines de coups plus tard.

Pour cela, nous avons implémenté plusieurs stratégies d'intelligence artificielle de niveaux croissants. Ces stratégies sont organisées dans le package fr.bordeaux.ai, lui-même subdivisé en plusieurs sous-packages reflétant la séparation des responsabilités :

— fr.bordeaux.ai.base — les abstractions communes à toutes les IA
(AiPlayer, HeuristicEvaluator)
— fr.bordeaux.ai.random — le joueur aléatoire
— fr.bordeaux.ai.heuristics — les trois fonctions d'évaluation de Minimax
— fr.bordeaux.ai.minimax — l'algorithme Minimax et son builder
— fr.bordeaux.ai.mcts — l'algorithme Monte Carlo Tree Search
— fr.bordeaux.ai.nn — le modèle de réseau de neurones
Toutes les stratégies partagent une base commune : la classe abstraite

AiPlayer, qui hérite elle-même de Player. Cette classe impose à chaque IA d'implémenter une seule méthode centrale : getNextMove(GameState state). C'est le point d'entrée unique par lequel le moteur de jeu demande à l'IA de choisir son prochain coup. Du point de vue du moteur, peu importe que l'IA soit aléatoire ou basée sur un réseau de neurones ; elle répond toujours via cette même méthode. C'est le principe du polymorphisme en programmation orientée objet.

Toute classe implémentant l'interface HeuristicEvaluator doit être capable d'analyser un état de jeu donné pour lui assigner une valeur ou un score numérique, ici elle impose que toutes les heuristiques aient une méthode Evaluate(GameState state). Cette interface permet de rendre les fonctions d'évaluation interchangeables à l'exécution, sans modifier le code de l'algorithme de recherche. C'est le patron de conception Strategy : on peut changer de stratégie d'évaluation comme on changerait de règle ; l'algorithme qui utilise cette règle n'a pas besoin de savoir laquelle est active. Par conséquent, l'interface Strategy est représentée par HeuristicEvaluator. Les classes AdvancedHeuristic, IntermediateHeuristic et SimpleHeuristic sont les implémentations concrètes de cette stratégie. Enfin, MiniMaxPlayer agit comme le contexte qui utilise la stratégie définie par HeuristicEvaluator.

Le Joueur Aléatoire (RandomPlayer) : Un Acteur Fondamental Avant d'explorer des algorithmes plus avancés, définissons le RandomPlayer. Il identifie tous les coups légaux possibles et en choisit un au hasard, avec une probabilité égale pour chaque option. Il est utilisé si MiniMax est interrompu par manque de temps pour atteindre la profondeur définie en paramètre.

Théorie Approfondie de l'Algorithme Minimax et Élagage Alpha-Bêta

L'Algorithme Minimax : Stratégie en IA pour les Jeux

L'algorithme Minimax est une méthode conçue pour déterminer la meilleure action dans les jeux à deux joueurs, déterministes et à information complète. Dans son implémentation, l'algorithme suppose que l'adversaire joue le meilleur coup à sa disposition. Il repose sur une analyse des possibilités de jeu, structurée sous la forme d'un arbre de décision.

Structure de l'Arbre de Jeu Minimax

1. Racine : L'état initial du plateau.
2. Branches : Chaque nœud se ramifie en branches représentant les coups légaux disponibles.

3. Feuilles : Aux extrémités de l'exploration, une fonction d'évaluation heuristique attribue un score numérique à la position (avantage pour le joueur IA, le joueur MAX).

Principe d'Alternance (MAX et MIN)

L'exploration de l'arbre alterne entre deux objectifs :

— Nœuds MAX (Tour de l'IA) : L'IA cherche à MAXimiser son score. Elle choisit le coup menant à l'état enfant avec la valeur heuristique la plus élevée.

— Nœuds MIN (Tour de l'Adversaire) : L'algorithme suppose que l'adversaire cherche à MINimiser le score de l'IA. Il simule le choix de l'adversaire menant à l'état enfant avec le score le plus bas.

Ce processus de remontée des valeurs (des feuilles à la racine) permet à l'IA de choisir le premier coup garantissant le meilleur résultat possible.

La complexité exponentielle du Minimax l'oblige à explorer des branches dont le calcul s'avère en réalité inutile, l'élagage Alpha-Bêta permet précisément de les ignorer.

1.2.2 Optimisation : L'Élagage Alpha-Bêta

L'élagage Alpha-Bêta est l'optimisation mathématique de référence pour le Minimax. Cette technique réduit le nombre de nœuds à explorer en ignorant les branches qui ne peuvent pas aboutir à un meilleur score que celui déjà garanti par une autre branche. L'élagage maintient l'optimalité du résultat avec un gain de vitesse considérable.

L'exploration exhaustive de l'arbre de jeu étant trop coûteuse en ressources, l'algorithme maintient deux bornes de coupe (des limites mathématiques permettant de stopper prématurément le calcul) :

— α : Le score minimum garanti pour le joueur MAX (l'intelligence artificielle).

— β : Le score maximum garanti pour le joueur MIN (l'adversaire).

Durant la descente dans l'arbre, l'algorithme compare constamment les options des deux joueurs, et l'exploration d'une branche est immédiatement interrompue (élaguée) dès que la condition $\alpha \geq \beta$ est validée. Pour comprendre logiquement cette mécanique, il faut considérer la valeur α comme l'assurance de l'intelligence artificielle (le joueur MAX) : c'est le score minimum qu'elle s'est déjà garanti en explorant une branche précédente. Lorsque l'algorithme commence à évaluer une nouvelle alternative (un autre coup possible pour MAX), il simule les réponses de l'adversaire (le joueur MIN). Si, au cours de cette simulation, l'algorithme découvre un coup pour MIN qui fait chuter l'évaluation à une valeur β inférieure ou égale à l'assurance α, le calcul s'arrête. En effet, le but de MIN étant de minimiser le score, il choisira à coup sûr cette réponse punitive, ou une pire encore s'il en trouve une en cherchant plus loin. Anticipant cela, MAX sait pertinemment que s'engager dans cette direction le forcera à subir ce score β, ce qui est strictement moins avantageux que son plan de secours α déjà sécurisé ailleurs. MAX ne choisira donc jamais d'emprunter ce chemin initialement. Par conséquent, il devient mathématiquement inutile de gaspiller du temps de calcul pour évaluer le reste de la branche, justifiant ainsi la coupe immédiate du nœud.

Cette coupe radicale permet de dédier la puissance de calcul aux branches viables, augmentant mécaniquement la profondeur d'analyse dans la limite de temps impartie.

1.2.3 L'Évaluation Heuristique : Le Moteur de Décision de l'IA

Lorsqu'un nœud atteint la profondeur maximale ou que le temps est écoulé, la partie n'est généralement pas terminée. L'algorithme fait alors appel à une fonction d'évaluation (heuristique) pour estimer la valeur tactique de cette position précise. Plus ce score est élevé, plus la situation est jugée favorable pour l'IA (MAX). Le projet propose trois niveaux d'évaluation :

L'Heuristique Simple (SimpleHeuristic) L'Heuristique Simple (SimpleHeuristic) se concentre sur l'objectif le plus rudimentaire du jeu, qui est d'avancer vers l'arrivée. D'un point de vue théorique, elle s'appuie sur une évaluation spatiale purement absolue en calculant le nombre de cases franchies en ligne droite par rapport à sa position de départ. Bien que cette méthode arithmétique soit extrêmement rapide à calculer, elle présente des limites tactiques majeures : elle reste totalement ignorante de son environnement puisqu'elle ne prend en compte ni les murs posés sur le plateau, ni la position actuelle de l'adversaire.

L'Heuristique Intermédiaire (MediumHeuristic) L'Heuristique Intermédiaire (MediumHeuristic) affine cette première approche en introduisant la notion d'avantage relatif. Plutôt que d'évaluer l'intelligence artificielle de manière isolée, elle reprend exactement la logique de l'heuristique simple et l'applique aux deux joueurs pour en extraire un différentiel. L'algorithme calcule le nombre de cases franchies en ligne droite par l'IA depuis sa position de départ, puis y soustrait cette même mesure effectuée pour l'adversaire. Cette évaluation reste strictement spatiale et calcule une distance directe, ignorant la présence éventuelle de murs sur le trajet. Un score positif indique ainsi clairement que l'IA est physiquement en tête de la course. Surtout, cette comparaison de distances incite naturellement l'algorithme à jouer de manière défensive. Si la pose d'un mur ralentit fortement la progression ennemie sans trop pénaliser celle de l'IA, l'écart mathématique entre les deux joueurs augmentera, validant ainsi la pertinence de ce blocage tactique.

L'Heuristique Avancée (AdvancedHeuristic) L'Heuristique Avancée (AdvancedHeuristic) constitue le véritable cerveau tactique du joueur Minimax. Elle calcule son score final en agrégeant plusieurs métriques dynamiques, c'est-à-dire des mesures chiffrées qui évoluent en temps réel selon la configuration du plateau, contrairement à des valeurs fixes. Ces métriques incluent l'analyse spatiale des obstacles, la gestion de la réserve de murs et la capacité à forcer des détours.

Tout d'abord, contrairement aux heuristiques précédentes basées sur des coordonnées brutes, elle effectue une analyse topographique concrète : elle étudie le relief du terrain créé par les murs. Elle calcule la vraie distance requise pour gagner en tenant compte du labyrinthe formé par les obstacles déjà posés, en utilisant l'algorithme de parcours en largeur BFS (Breadth-First Search). Ce dernier explore systématiquement toutes les cases accessibles pour garantir le calcul du chemin le plus court.

Ensuite, elle intègre une gestion des ressources en comparant le nombre de murs non joués restants dans la main de l'IA par rapport à ceux de l'adversaire. Cet avantage matériel est modulé par un coefficient fluctuant (wallCoeff). Ce coefficient traduit le fait qu'un mur conservé en réserve en début de partie rapporte beaucoup de points en raison de son fort potentiel de blocage, mais que sa valeur tactique (son utilité réelle pour gagner) diminue à mesure que l'IA approche de la victoire et que les opportunités de pose se raréfient.

Le cœur de cette heuristique repose sur la stratégie de contournement, c'est-à-dire l'art d'obliger l'adversaire à s'éloigner de son chemin le plus direct. Pour évaluer la réussite de cette stratégie, l'algorithme calcule un bonus de détour en comparant la distance théorique de l'adversaire sans aucun obstacle (la distance de Manhattan, qui est la somme des différences absolues de coordonnées) avec sa distance réelle trouvée par le parcours BFS. La différence entre ces deux valeurs représente le détour exact imposé par les murs placés sur le plateau. L'IA est ainsi lourdement récompensée (WALL_BONUS) si elle parvient à créer des goulots d'étranglement obligeant l'adversaire à emprunter de longs chemins sinueux.

Enfin, l'algorithme gère les conditions de victoire de manière prioritaire pour éviter que des calculs secondaires ne masquent un gain ou une perte imminente. Dans l'implémentation, le seuil de victoire est une constante nommée WIN_THRESHOLD fixée à la valeur 1, représentant la distance minimale, calculée en nombre de pas par l'algorithme BFS, séparant un pion de sa ligne d'arrivée. Ce seuil est déterminé dynamiquement à chaque évaluation : l'IA lance un parcours de graphe pour elle-même et pour son adversaire afin de connaître le nombre précis de cases restant à parcourir en tenant compte des murs. Si le résultat est inférieur ou égal à 1, cela signifie que le joueur est soit déjà sur sa ligne d'arrivée, soit à une seule case de celle-ci. Dès que cette proximité extrême est détectée, la fonction interrompt immédiatement l'agrégation des autres métriques, comme la réserve de murs ou les bonus de détour, car elles n'ont plus d'importance tactique face à une fin de partie immédiate. L'algorithme renvoie alors un score absolu de 10 000 pour une victoire de l'IA ou de -10 000 pour une victoire adverse, garantissant que l'intelligence artificielle privilégiera toujours le coup menant au succès final plutôt qu'une simple amélioration stratégique.

Structures de Données et Optimisations (Minimax) L'implémentation repose sur des structures de données spécifiquement choisies pour limiter l'empreinte mémoire (la quantité de mémoire vive RAM consommée) et accélérer la vitesse d'exécution :

- File à double entrée (Deque) pour l'exploration de l'arbre : Afin d'éviter l'instanciation massive (la création répétitive et très coûteuse de nouveaux objets en mémoire pour chaque scénario), la classe MinimaxPlayer ne clone pas le plateau de jeu pour chaque branche. Elle maintient l'historique de la branche courante dans une structure de pile (Deque<Move>). L'algorithme procède par mutation directe (modification de l'état unique du plateau) grâce à un mécanisme de backtracking (retour sur ses pas) : on simule un coup pour descendre dans l'arbre, puis on le dépile pour l'annuler et remonter.
- Interface Fonctionnelle (BiFunction) : Plutôt que de vérifier à chaque coup quel algorithme utiliser via une structure de contrôle classique (type if/else), le constructeur de MinimaxPlayer fige le comportement de l'IA dès son initialisation. Il assigne la méthode de résolution choisie — Minimax standard ou Approfondissement Itératif — à un attribut nommé mode. Lors de l'exécution, l'IA appelle directement cette fonction stockée, ce qui simplifie le flux de calcul en éliminant les branchements logiques répétitifs au cœur de la prise de décision.
- Table de hachage sécurisée (ConcurrentHashMap) : Les paramètres internes de l'IA (profondeur, temps, heuristique) sont exportés via la méthode getSettings à l'aide d'une ConcurrentHashMap (un dictionnaire stockant des paires clé-valeur). Cette structure est spécifiquement conçue pour être thread-safe (protégée contre les crashs si plusieurs processus ou éléments de l'interface graphique tentent de la lire en même temps).
- Listes dynamiques (List) pour le calcul heuristique : Dans la classe AdvancedHeuristic, l'évaluation tactique ne s'arrête pas aux coordonnées brutes. Le calcul des distances réelles s'appuie sur une liste pour stocker la séquence des identifiants des cases validées par l'algorithme BFS (Breadth-First Search, un parcours de graphe en largeur garantissant de trouver le chemin le plus court tout en contournant les murs existants).

Bien que l'algorithme Minimax soit redoutable, son exploration systématique montre ses limites face à des situations présentant un fort facteur de branchement, où le nombre de coups possibles devient beaucoup trop grand pour être calculé dans le temps imparti. C'est pour pallier cette explosion combinatoire qu'intervient l'algorithme Monte

Carlo Tree Search (MCTS).

Mécanique de l'Algorithme Monte Carlo Tree Search Contrairement au Minimax qui tente d'évaluer toutes les éventualités, le MCTS ne vérifie pas toutes les possibilités. Il s'agit d'une approche probabiliste, signifiant qu'elle fait intervenir l'aléatoire dans ses choix de simulation, et asymétrique, car elle concentre sa puissance de calcul pour explorer en profondeur les branches les plus prometteuses plutôt que de parcourir l'arbre de manière uniforme. Le processus itératif s'exécute en boucle selon quatre phases distinctes orchestrées par la méthode getBestMoveAfterSearch dans la classe MctsNode.

La première phase est la sélection, étape au cours de laquelle l'algorithme amorce sa recherche en descendant dans l'arbre depuis la racine. Le but de cette descente est de trouver le candidat idéal pour poursuivre l'exploration. L'algorithme obéit à une règle de priorité stricte dictant le passage à l'étape suivante. En effet, tant qu'un état a déjà testé l'intégralité de ses coups possibles et possède des nœuds enfants, la phase de sélection continue de s'enfoncer dans l'arbre. À chaque intersection, l'intelligence artificielle choisit de descendre vers le nœud enfant le plus "prometteur", c'est-à-dire celui qui maximise le score évalué par la méthode bestUCT. Ce score prometteur désigne le meilleur compromis mathématique calculé entre l'exploitation d'une branche affichant déjà un fort taux de victoire et l'exploration nécessaire des chemins encore peu visités, le score UCT est expliqué plus en détail après.

Cependant, dès que l'algorithme croise au cours de sa descente un nœud disposant encore d'au moins un coup légal non essayé (une condition vérifiée dynamiquement par la méthode hasUntriedLegalMoves), la phase de sélection s'arrête net. Ce nœud partiellement exploré devient alors la cible prioritaire absolue pour la phase suivante, garantissant ainsi que l'arbre s'élargisse systématiquement pour évaluer toutes les options initiales d'une position avant de se creuser aveuglément en profondeur.

Vient ensuite la phase d'expansion. Si le nœud sélectionné n'est pas un état de fin de partie, un coup non testé est extrait pour créer un nouveau nœud enfant dans l'arbre de recherche. Pour implémenter cela efficacement, la classe MctsNode s'appuie sur des listes dynamiques. Lors de sa création, chaque état génère et stocke ses coups potentiels dans une liste dédiée nommée untriedMoves. Afin de garantir la stochasticité de l'algorithme, c'est-à-dire l'introduction d'un hasard mathématique visant à neutraliser l'ordre prévisible dans lequel le jeu génère initialement les actions, comme par exemple regarder d'abord les coups de pion avant la pose de murs, cette collection est immédiatement mélangée grâce à la fonction Collections.shuffle. Ainsi, toutes les options tactiques bénéficient d'une probabilité strictement égale d'être explorées, ce qui empêche l'intelligence artificielle de développer des angles morts stratégiques et d'orienter artificiellement les statistiques de l'arbre de recherche.

Une optimisation majeure de cette phase d'expansion réside dans la vérification paresseuse des coups. Dans un jeu comme Quoridor, calculer la légalité absolue de toutes les poses de murs possibles est extrêmement lourd en ressources, car il faut lancer des algorithmes de parcours de graphe pour s'assurer qu'aucun joueur n'est définitivement bloqué. Pour éviter ce goulet d'étranglement, la création du nœud génère uniquement des coups dits "pseudo-légaux" (des coups respectant les règles de base mais sans vérification approfondie des blocages). Ce n'est qu'au moment précis où l'algorithme s'apprête à étendre l'arbre, via la méthode hasUntriedLegalMoves, qu'il vérifie concrètement si le dernier coup de la liste mélangée respecte la totalité des règles via le RuleChecker. Si le coup s'avère illégal, il est simplement retiré de la liste et l'algorithme évalue le suivant dans le dessous de la pile. Cette approche économise énormément de temps de calcul en ignorant la vérification stricte des centaines de coups potentiels qui ne seront de toute façon jamais piochés par le MCTS dans le temps imparti.

Par ailleurs, l'algorithme gère de manière proactive le scénario où le coup tiré lors de l'expansion mène directement à une victoire. Si le nouvel état généré est identifié comme une fin de partie, il devient mathématiquement inutile de gaspiller du temps de calcul pour y lancer des simulations aléatoires. Le nœud est alors explicitement verrouillé et l'algorithme court-circuite l'étape suivante pour passer directement à la phase de rétropropagation. La branche victorieuse est ainsi figée, ce qui permet à la puissance de calcul de se concentrer exclusivement sur l'évaluation des autres alternatives.

Dans le cas contraire, si l'expansion aboutit à un état en cours de jeu, l'algorithme enclenche la troisième phase : la simulation (couramment appelée Playout). À partir de ce nouveau nœud, l'algorithme simule une fin de partie en jouant une série de déplacements purement aléatoires jusqu'à déterminer un vainqueur ou atteindre une limite de sécurité fixée à 200 tours.

Enfin, la rétropropagation (Backpropagation) clôture le cycle en remontant le résultat de cette simulation (ou de la victoire immédiate) le long de la branche explorée afin de mettre à jour les statistiques mathématiques de chaque nœud parent traversé jusqu'à la racine. Concrètement, chaque nœud de l'arbre conserve en mémoire deux variables entières distinctes : son nombre total de visites et son nombre de victoires accumulées. Lors de cette phase de remontée, l'algorithme incrémente systématiquement de un le compteur de visites de chaque nœud parent rencontré. Parallèlement, il compare l'index du joueur gagnant de la simulation avec celui de l'intelligence artificielle : si l'IA est victorieuse, le compteur de victoires de ces mêmes nœuds est également incrémenté. Ces deux données sont fondamentales, car leur division permet de calculer le taux de victoire brut d'une branche, une métrique indispensable qui sera ensuite réinjectée dans la formule mathématique UCT lors de la phase de sélection des itérations suivantes

pour guider les futurs choix de l'algorithme.

Exploration et Exploitation : Le Calcul UCT La véritable intelligence du MCTS réside dans sa phase de sélection, pilotée par la classe MctsPlayer. Bien que cette classe puisse dynamiquement déléguer ce choix à un réseau de neurones artificiels (une approche détaillée dans la section suivante), nous nous concentrerons ici sur sa méthode d'évaluation classique nommée selectBestChildUsingUCT. Pour choisir le chemin à emprunter dans l'arbre, l'algorithme utilise la formule mathématique UCT (Upper Confidence bounds applied to Trees). Cette équation garantit un compromis rigoureux entre l'exploitation et l'exploration.

Mathématiquement, l'algorithme calcule pour chaque nœud enfant un score composé de deux parties. La première partie est le taux de victoire brut, obtenu en divisant le nombre de victoires par le nombre de visites de ce nœud. Cela représente l'exploitation, signifiant concrètement le fait de privilégier les coups ayant déjà prouvé leur efficacité. La seconde partie est le terme d'exploration, calculé en multipliant une constante fixée à 1.41 par la racine carrée du logarithme des visites du nœud parent divisé par les visites de l'enfant. Ce terme d'exploration augmente artificiellement le score des branches qui ont été peu visitées. Ainsi, l'algorithme est naturellement poussé à vérifier les coups ignorés pour s'assurer de ne rater aucune stratégie cachée, tout en continuant d'approfondir les lignes de jeu majoritairement gagnantes.

Les Limites de la Simulation Aléatoire et l'Introduction au Machine Learning Si la méthode UCT standard est très performante pour diriger la recherche, elle repose fondamentalement sur la phase de simulation (le Playout aléatoire) pour évaluer la qualité d'un nœud. Dans un jeu complexe, jouer des coups au hasard jusqu'à la fin de la partie peut aboutir à des résultats chaotiques qui ne reflètent pas la véritable valeur stratégique d'une position. Un simple mur placé au hasard peut fausser toute l'évaluation. Pour pallier cette limite, l'intelligence artificielle introduit un modèle d'apprentissage automatique supervisé. La classe MctsPlayer agit alors comme un aiguilleur paramétrable offrant la possibilité de configurer le moteur de recherche selon deux modes exclusifs, c'est-à-dire en utilisant soit l'approche UCT traditionnelle, soit le réseau de neurones. Lorsque l'apprentissage automatique est sélectionné, le modèle neuronal évalue instantanément les chances de victoire de la position en cours au lieu de se fier au calcul mathématique classique. Cette approche remplace ainsi avantageusement la dépendance à la lourde phase de simulation aléatoire par une prédiction déterministe et stratégiquement éclairée. Le terme déterministe certifie que pour une configuration strictement identique du plateau, le réseau de neurones renverra toujours exactement le même score de victoire, puisqu'il remplace le hasard des simulations par un calcul matriciel aux paramètres fixes. Parallèlement, cette prédiction est qualifiée de stratégiquement éclairée car elle ne se base plus sur des déplacements effectués à l'aveugle. Le modèle analyse concrètement la qualité de la position en évaluant la véritable distance séparant les joueurs de leur ligne d'arrivée et en comptabilisant le nombre de murs restants dans les réserves, appliquant ainsi une réelle compréhension tactique acquise lors de son entraînement.

L'Architecture du Neurone et la Gestion des Tenseurs L'évaluation neuronale est encapsulée dans la classe NeuralNetworkEvaluator, qui sert d'interface avec le moteur de calcul TensorFlow. Ce modèle s'appuie spécifiquement sur une architecture de type régression logistique, ce qui correspond structurellement à l'utilisation d'un unique neurone artificiel.

Le fonctionnement de ce neurone repose sur une équation mathématique fondamentale où la prédiction finale est le résultat de l'opération $P = \sigma(W \cdot X + b)$. Dans cette formule, la variable X représente les caractéristiques en entrée fournies par le jeu. La variable W désigne les poids de la matrice, représentant l'importance accordée par l'intelligence artificielle à chaque paramètre. Enfin, la variable b correspond au biais, un ajustement numérique global indépendant des entrées.

Le système prend en entrée quatre caractéristiques mathématiques précises extraites de la partie en cours. Il calcule d'abord la véritable distance séparant le joueur de sa ligne d'arrivée en effectuant un parcours de graphe BFS. Pour que cette donnée soit exploitable par le réseau de neurones, elle doit être normalisée. L'algorithme divise donc ce nombre de pas par la distance maximale théorique du plateau, puis inverse mathématiquement ce résultat. Ainsi, une distance nulle produit la valeur maximale de 1.0 pour indiquer une victoire atteinte, tandis que l'éloignement maximal donne un résultat de 0.0. Il intègre ensuite la distance de l'adversaire, elle aussi obtenue par un algorithme BFS et divisée par la distance maximale. Il est important de noter que cette valeur adverse n'est pas inversée, car pour l'intelligence artificielle, un résultat de 1.0 indiquant que l'ennemi est bloqué très loin constitue une situation hautement favorable. Les troisième et quatrième paramètres sont respectivement le nombre de murs restants pour l'IA et pour son adversaire, tous deux divisés par leur capacité initiale de dix (si la capacité initiale est supérieure à 10, on s'assure de rester dans l'intervalle [0,1] en prenant 1.0 au maximum) . Cette dernière opération permet de ramener l'intégralité des données fournies au neurone sur une échelle mathématique parfaitement homogène comprise entre zéro et un.

L'algorithme de TensorFlow applique alors une multiplication matricielle entre ces quatre entrées X et leurs poids respectifs W, additionne le biais b, puis fait passer le résultat brut par la fonction d'activation sigmoïde désignée par le symbole σ. Cette fonction prend la forme d'une courbe en "S" écrasant les valeurs numériques pour forcer un résultat

strictement compris entre 0 et 1, représentant ainsi de façon lisible le pourcentage direct de chances de victoire de la position évaluée.

Techniquement, pour interagir avec le cœur natif de TensorFlow, les données primitives Java sont systématiquement converties en tenseurs de type TFloat32. Ce sont des matrices multidimensionnelles hautement optimisées pour être manipulées très rapidement par le moteur C++ sous-jacent. De plus, le graphe computationnel modélisant ces opérations mathématiques et la session d'exécution sont maintenus ouverts en mémoire vive. Cette architecture évite le coût de latence massif qu'impliquerait une recompilation du modèle à chaque nouvelle inférence requise par le MCTS. Pour gérer rigoureusement ces ressources externes, la classe implémente l'interface native AutoCloseable. Cette sécurité garantit la destruction explicite des objets alloués en C++ à la fermeture du programme, prévenant toute fuite de mémoire (Memory Leak) puisque ces objets échappent par nature au ramasse-miettes (Garbage Collector) de Java.

L'Entraînement Supervisé et la Descente de Gradient Pour que ce neurone artificiel et ses poids internes soient capables d'évaluer correctement une position, il faut d'abord les ajuster lors d'une phase d'apprentissage autonome. La classe NeuralNetworkTrainer est responsable de cet entraînement supervisé. Elle génère d'énormes quantités de données en faisant s'affronter l'IA contre elle-même (Self-Play) au travers de milliers de parties simulées. Durant ces simulations de génération de données, l'algorithme applique une politique d'exploration appelée Epsilon-Greedy. Une probabilité fixée à dix pour cent force ponctuellement l'IA à jouer des coups totalement aléatoires. Le reste du temps, le système ne cherche pas à calculer le meilleur mouvement tactique absolu de la position, mais délègue plutôt la décision à un algorithme Minimax intentionnellement bridé à une profondeur de recherche d'un seul tour. Ce choix technique est un compromis indispensable pour garantir la rapidité des simulations, puisqu'une recherche complète et profonde prendrait un temps incalculable pour générer les milliers de parties requises par l'apprentissage. Cette mécanique hybride empêche finalement l'algorithme de tourner en boucle sur les mêmes séquences d'ouverture et le force à découvrir de nouvelles configurations complexes du labyrinthe tout en générant des données de jeu cohérentes.

Une fois ces parties terminées et les résultats finaux connus, le réseau compare ses propres prédictions passées avec la réalité des victoires et des défaites. Cette différence est mesurée avec une grande précision par une fonction mathématique nommée Entropie Croisée Binaire, ou Binary Crossentropy. Cette fonction de perte calcule l'écart entre la probabilité prédite par la sigmoïde et le label réel de la partie, valant 1.0 pour une victoire observée et 0.0 pour une défaite. La particularité mathématique de l'Entropie Croisée Binaire est d'appliquer une pénalité logarithmique extrêmement sévère si le modèle se trompe en étant très confiant, par exemple s'il prédisait 90 pour cent de chances de victoire pour une position qui a finalement mené à un échec.

Afin de corriger cette erreur de prédiction, l'algorithme fait appel à un optimiseur basé sur la méthode de la Descente de Gradient. Ce mécanisme mathématique calcule la dérivée de l'erreur pour déterminer dans quelle direction modifier les variables internes. Il ajuste alors de manière infime les poids synaptiques W et le biais b du neurone en fonction d'un taux d'apprentissage défini, afin que lors de la prochaine situation similaire, la prédiction se rapproche naturellement de la réalité. En répétant ce cycle d'évaluation et de correction sur des dizaines de milliers d'exemples répartis sur plusieurs centaines d'époques d'entraînement, le modèle met progressivement à jour sa matrice de poids pour affiner sa compréhension tactique du jeu.

2 Architecture du projet

2.1 Justification du choix d'architecture

Comme décrit dans le cahier des charges, notre projet qui consiste a implementer le jeu quoridor en java decrit au ci-haut doit respecter l'ensemble des règles du jeu Quoridor. De plus, il nous est demandé de mettre en œuvre trois modes d'interaction : ligne de commande (CLI), interface graphique (GUI) et jeu en réseau, ainsi que plusieurs intelligences artificielles (Minimax et Monte Carlo Tree Search).

Pour répondre à tous ces exigences, nous avons opté pour une architecture modulaire orientée domaine, inspirée de la Clean Architecture[2].

La Clean Architecture repose sur un principe fondamental : la séparation entre la logique métier et les détails d'implémentation. Dans ce modèle : Le cœur domaine contient toutes les entités métier de l'application. Il est totalement indépendant. Les adaptateurs (CLI, GUI, réseau, persistance, IA) interagissent avec le cœur via des interfaces abstraites et des implémentations concrètes. par cette architecture nous garantisons la cohérente du logique métier, on peut facilement ecrire des teste, ajouter de nouvelles IA ou interfaces et nous sommes sur de la robustesse de notre jeu, car toutes les actions passent par la validation du cœur métier et en fin on peut facilement Maintenir notre jeu, car les modifications sur l'interface n'impactent pas le cœur.

2.2 Vue d'ensemble des packages

Voici les onze packages distincts de notre projet, chacun ayant une responsabilité bien définie que les decrivons de manieres brefs ici :

Package	**Responsabilité**
core	Entités métier : Board, GameState, Player, Move, Position, GameEngine, RuleChecker
Main	Point d'entrée, résolution du mode de lancement, patron Factory
cli	Interface en ligne de commande avec le pattern Command pour chaque action utilisateur
gui	Interface graphique comportant les menus 'File' et 'Game' (JavaFX), proposant des raccourcis clavier. L'utilisateur doit pouvoir définir ses propres raccourcis pour chaque action listée
network	Implémente les commandes réseau de base et étendues pour gérer plusieurs clients simultanément, en utilisant un thread par client ou de la programmation asynchrone. Serveur TCP avec découverte UDP et client réseau
ai	Joueurs IA : Random, Minimax (alpha-beta, iterative deepening), MCTS, réseau de neurones
persistence	Sérialisation/désérialisation de l'état de jeu vers un fichier texte
util	Logger singleton partagé dans toute l'application
i18n	Gestion de la langue du logiciel. Si les variables d'environnement 'LANG' ou 'LC_ALL' sont définies, le programme doit les utiliser pour déterminer la langue à utiliser
graph	Module Graph et GraphBuilder implémenté sous la forme d'une liste d'adjacence (adjacency list) pour représenter l'état du plateau de jeu
config	Fichier de configuration nommé '.quoridorrc', placé à la racine du répertoire de l'utilisateur (HOME). Le fichier est lu au démarrage du programme s'il est présent, et les options qu'il contient sont utilisées par défaut. Sinon, un fichier de configuration minimal est créé automatiquement

TABLE 1 – Vue d'ensemble des packages

2.3 Vue en couches concentriques

Dans cette section nous vous proposons notre archichesture global en couches concentriques. les dependances entre packages ne doivent pointer que vers l'interieur du systeme. Le module **core** qui contient les entites metier — ne connait aucun autre module. C'est **network**, **gui** ou **cli** qui dependent de **core**, jamais l'inverse. Cette contrainte est visible dans le diagramme.

schema en couches concentriques de notre architecture du projet (Fig. 1).

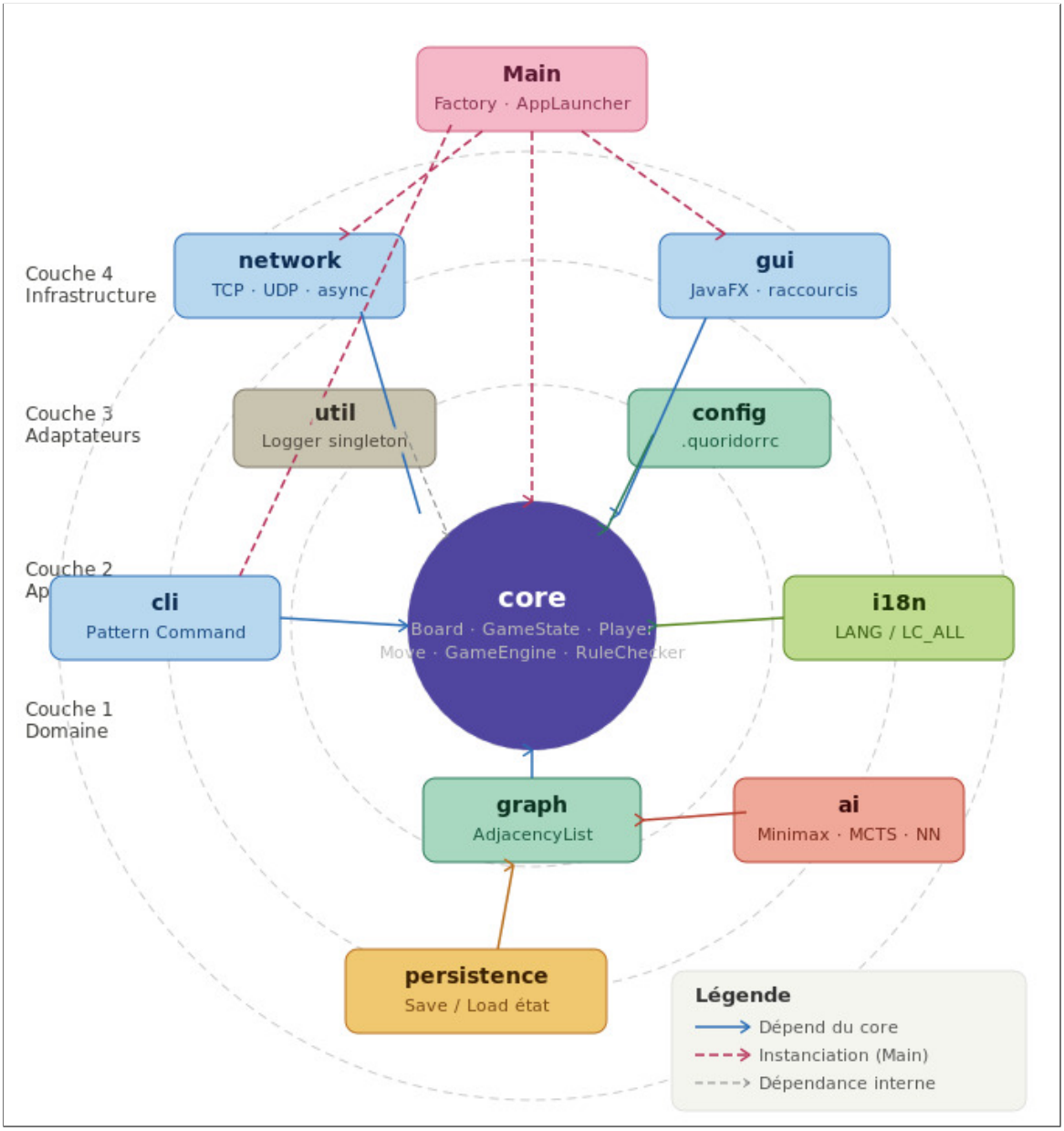


FIGURE 1 – Une architecture en couches

2.4 Couche Domaine — Package `core`

Le module **core** est le coeur du systeme. Il encapsule l'integralite de la logique metier du jeu et n'a aucune dependance externe. Le **RuleChecker** appelle le module **graph** pour verifier qu'apres la pose d'un mur, chaque joueur dispose encore d'un chemin vers son bord cible.

voici un diagramme indique comment son lié les differents classes du package core.

schema du couche core de notre architecture(Fig. 2).

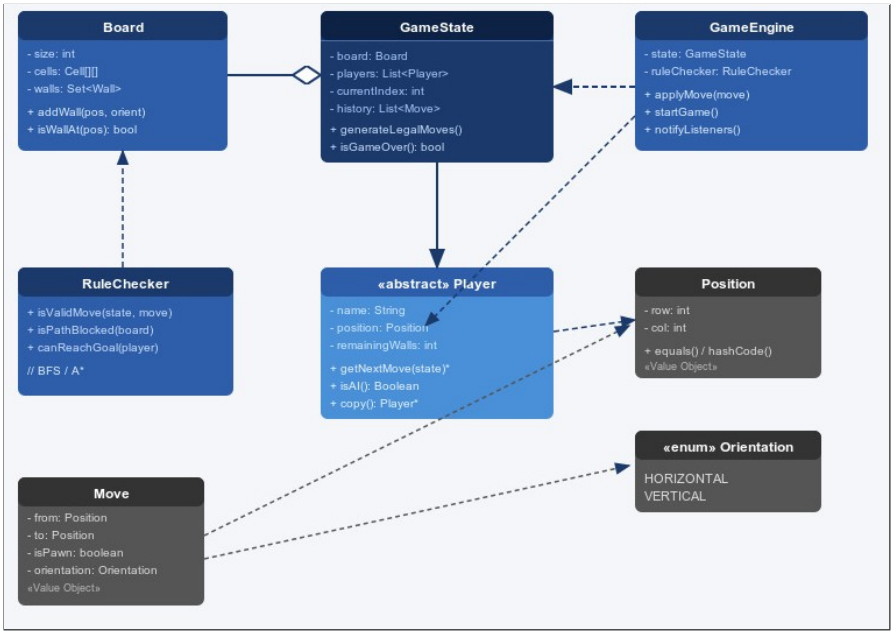


FIGURE 2 – Une architecture en couches

2.5 Couche Principale — Package `Main`

Le module **Main** joue le role de **composition root** : il instancie tous les autres modules et cable leurs dependances. Il utilise le pattern **Factory** via *AppLauncher* pour creer les adaptateurs selon le mode de lancement.

Au demarrage, le Main lit config (.quoridorrc) pour determiner le mode de jeu. Il instancie network si une partie reseau est demandee, gui pour le mode graphique, cli pour le mode terminal. Il injecte les dependances dans chaque module sans que ceux-ci se connaissent mutuellement.

voici un diagramme indique comment son lié les differents classes du package main.

schema du couche main de notre architecture (Fig. 3).

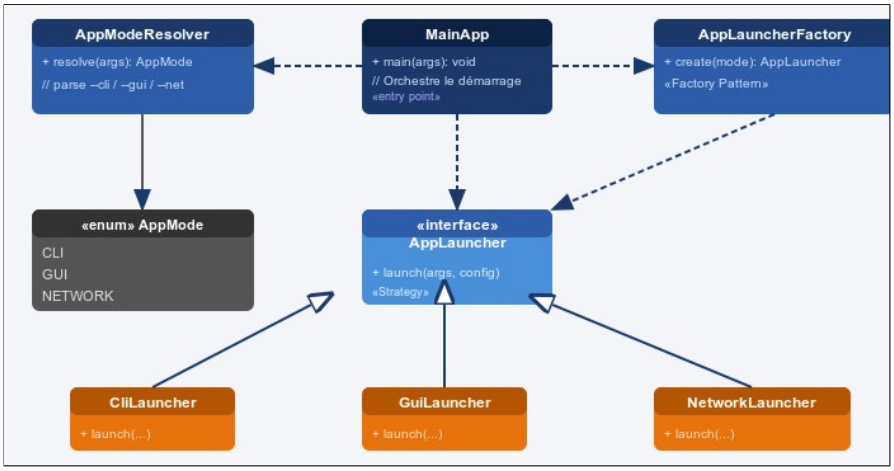


FIGURE 3 – Une architecture en couches

2.6 Interface CLI — Pattern Command

Le module **cli** implemente le pattern **Command** : chaque commande saisie est encapsulee dans un objet qui connait son action et peut etre annulee. Ce pattern est adapte a Quoridor car il permet de rejouer ou d'annuler l'historique des coups. Notre cli est instancie par Main au meme titre que gui et network. Il lit les entrees console, les parse, construit les objets Command correspondants et les execute sur *GameEngine*. Le plateau est affiche en ASCII en sortie standard.

voici un diagramme indique comment son lié les differents classes du package cli.

schema du couche cli de notre architecture (Fig. 4).

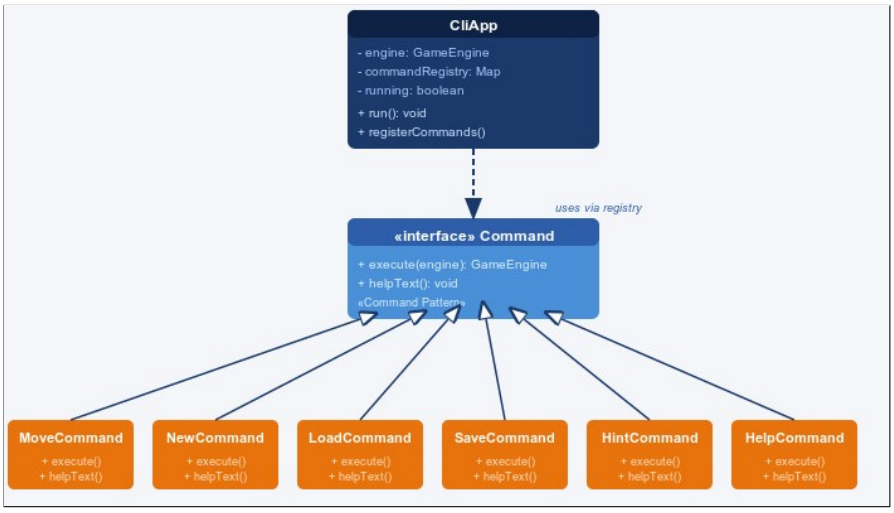


FIGURE 4 – Une architecture en couches

2.7 Points forts architecturaux

- **Indépendance du domaine** : le package `core` est du Java pur, sans dépendance externe
- **Extensibilité de l'IA** : tout nouvel algorithme étend simplement `AiPlayer`
- **Pattern Command en CLI** : chaque commande est testable et extensible indépendamment
- **Persistance découplée** : `GameSaver` / `GameLoader` ne connaissent que les interfaces du `core`
- **Builder pour MinimaxPlayer** : configuration déclarative et lisible de l'IA

2.8 Axes d'amélioration

Références

[1] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.

[2] Robert C. Martin. *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall, Boston, MA, USA, 2017.

A Section en annexe

Besoins	Titre du besoin	Description du besoin	État
F1	Lancement	Lancement via CLI avec options (-h, -V, -v, -d) et gestion des flux stdout/stderr.	FAIT
F2	Configuration	Lecture et création du fichier .qoridorrc au format INI dans le HOME.	FAIT
F3	i18n	Support de l'anglais et du français via les variables LANG/LC_ALL.	FAIT
F4	Interface CLI	Démarrage auto, application des options ou chargement d'une sauvegarde.	FAIT
F5	Mode Blitz	Mode avec temps limité par joueur (-b) et réglage de la durée (-t).	FAIT
F6	Mode Contest	Lecture d'un fichier de position et réponse du meilleur coup sur stdout.	FAIT
F7	Interface Graphique	Lancement d'une fenêtre GUI via l'option -g ou –gui.	FAIT
F8	Mode IA	Remplacement d'un ou plusieurs joueurs par une intelligence artificielle.	FAIT
F9	Options Spécifiques	Paramétrage du nombre de joueurs (2-4), des murs et de la taille du plateau (3-15).	FAIT
F10	Gestion des règles	Arbitrage du jeu, validation des coups, calcul des scores et victoire.	FAIT
F11	Erreurs Utilisateur	Gestion robuste des mauvaises saisies avec messages explicites.	FAIT
F12	Undo/Redo	Historique des coups et possibilité d'annuler/rétablir pour les humains.	FAIT
F13	Conseils	Système d'aide suggérant le meilleur coup à jouer selon l'état actuel.	FAIT
F14	Gestion Blitz	Affichage du chronomètre, gestion de la pause et défaite au temps.	FAIT
F15	Shell interactif	CLI sous forme de shell avec commandes (new, save, load, show, etc.).	FAIT
F16	Sauvegarde/Quitter	Confirmation de sauvegarde lors de la fermeture si la partie est modifiée.	FAIT
F17	Édition ligne	Support de l'édition de ligne (curseur) via Readline ou équivalent.	FAIT
F18	Historique Shell	Navigation dans les anciennes commandes et recherche par le caractère "+".	FAIT
F19	Complétion	Autocomplétion des commandes via la touche Tabulation.	FAIT

TABLE 2 – Vue d'ensemble des besoins et leurs realisations

F20	Notation des coups	Représentation ASCII du plateau et syntaxe de mouvement (ex : e2-e3, e2v).	FAIT
F21	Format Sauvegarde	Import/export en format texte ASCII avec paramètres, état et historique.	FAIT
F22	Commentaires	Support des commentaires mono-ligne (#) et bloc ({ }) dans les sauvegardes.	FAIT
F23	Paramètres Jeu	Format clé-valeur pour les réglages dans le fichier de sauvegarde.	FAIT
F24	Format Plateau	Représentation visuelle simplifiée du plateau pour la restauration.	FAIT
F25	Format Historique	Liste chronologique des tours et des coups par joueur.	FAIT
F26	Module Graph	Utilisation d'une liste d'adjacence et algorithmes de graphes indépendants.	FAIT
F27	Algos Graphes	Calcul des coups valides, scores et détection de fin de partie via le graphe.	FAIT
F28	Menus GUI	Implémentation des menus 'File' et 'Game' dans l'interface graphique.	FAIT
F29	Raccourcis Clavier	Raccourcis Ctrl+N, S, O, etc. et personnalisation via configuration.	FAIT
F30	Drag'n'Drop	Manipulation des pièces et des murs à la souris dans le GUI.	FAIT
F31	IA : Temps borné	Limite de réflexion de l'IA (5s par défaut) avec coup par défaut si dépassé.	FAIT
F32	IA : Minimax	Implémentation de l'algorithme Minimax avec élagage Alpha-Beta.	FAIT
F33	IA : Fonctions Score	Au moins 3 fonctions d'évaluation différentes pour l'IA Minimax.	FAIT
F34	IA : Deepening	Mode approfondissement itératif pour optimiser la recherche.	FAIT
F35	IA : Profondeur	Option pour fixer la profondeur max de recherche de l'algorithme.	FAIT
F36	IA : MCTS	Algorithme Monte Carlo Tree Search avec fonction UCT.	FAIT
F37	IA : Machine Learning	Fonction de sélection MCTS basée sur l'apprentissage automatique (ML).	FAIT
F38	Réseau : Basique	Découverte UDP, connexion TCP, ping/pong et client/serveur simple.	FAIT
F39	Réseau : Multi-clients	Serveur gérant plusieurs parties, scoreboard et validation anti-triche.	NON FAIT
F40	Réseau : Invitations	Système avancé d'invitations (accept/decline) et gestion des statuts joueurs.	NON FAIT

TABLE 3 – Vue d'ensemble des besoins leurs realisations